

Designing Web APIs

Learning Sources

[Designing Web APIs - Building APIs That Developers Love \(Jin, Sahni, & Shevat, 2018\)](#)

Notes

-Notes originally created in Dec 2024

Intro

-APIs are a core of modern development. They let you expose millions of developers to your code, who can then interact with it and innovate with it.

-By using, and exposing APIs, you can easily re-use existing code.

-During the early 90's, many tried to create a standardization for that could be used for APIs widely, but they were overly complex and sometimes required partial installation of the remote service.

-In the late 90's, standardization got better, first with SOAP (Simple Object Access Protocol) and XML (Extensible Markup Language), then REST (Representative State Transfer) and JSON (JavaScript Object Notation).

-APIs allow integration of a company's service into the service of someone else. That can often mean money for the company, for charging a fee to use the API, and the service it provides, or to promote integration of the company's product into the product of someone else, for a service fee.

Book Organization

- Theory - Ch 1-3 - The concepts of building an API, different API patterns, what makes a good API.
- Practice - Ch 5-7 - How to actually design an API and manage its production operations.
- Developer Love - Ch 8-11 - How to build a thriving ecosystem around your API.

Ch 01: What's an API?

-Application Programming Interface - The interface a software program presents to other programs, humans, and the internet.

-APIs are the building blocks for interoperability across the web. They create and manage accounts and orders. They forecast the weather. They process your credit card payments. They let you buy a pizza.

-APIs give a business a platform to spread their services to everyone.

Why APIs?

-APIs largely evolved from the need for companies to use the code of other companies, to not always need to re-invent the wheel (ex. Google Maps API).

Who Uses APIs?

-It is important to understand who the users of the API will be, what their needs are, and why they are using the API. Proper stakeholder input and requirement makes an API people actually want to use and do use.

-Get approval on the design for your API, from stakeholders, before you implement it. The cost of switching API designs after already making an unsuitable API is high.

-Example API Uses Cases:

- Lisa is a web dev that sells art and creates an API for artists to upload and show their work
- Ben is a back-end dev and creates an API to process orders for an e-commerce site
- Jane is a front-end dev who uses an API to integrate live customer support into her site

The Business Case for APIs

-Direct Monetization of APIs - via profit share models, subscription fees, or usage fees

-For integrating third party services into your product

-To generate leads or generate distribution channels for a product

-An API must be aligned with the core business, as a SaaS product is. Consumer facing APIs *must* work well.

-If an API detracts too much from the core business model of a business, be careful of implementing it, as it could distract people away from your profit (ex. Twitter API).

APIs for Internal Devs First, External Devs Second

-Even if API mostly built for internal devs, exposing as external API could still bring in demand for the company's product, or enable other companies to build products the company itself does not want to build.

-Ex. Slack API was originally for Slack's web, desktop, and mobile clients to display a messaging interface. Originally created for internal devs, Slack then built out external API, for developers to integrate it into their own apps. This allowed existing company software to easily integrate slack into existing systems.

-An advantage of releasing a first internal API to then be external facing is that it has already been well tested by internal use.

-Disadvantage of making internal external is that needs across the two may diverge, thus making needs less compatible.

APIs for External Devs First, Internal Devs Second

-Ex. Github - audience was initially mostly external developers. Business model thus built external API as first priority. Created API that serves both individuals and teams.

-When external devs come first, much less risk of internal first becoming too off-putting for external uses

APIs as the Product

-Stripe is an example. It provides a payment processing API for its product.

-In this case, the API must be fully seamless for customers. The entire business is aligned around it.

What Makes an API Great

-Mostly, does the API achieve its purpose and requirements?

-A good API will be easy to understand, usable, consistent, well documented, flexible, powerful, scalable, well performing, and changeable.

Ch 02: API Paradigms

-An API paradigm defines the interface exposing the API back-end. Not doing this can result in a change resistant, inflexible API. Plan before building.

2.1 Request-Response APIs

-Interface is exposed through an HTTP-based web server. Clients make HTTP requests to endpoints and server returns responses. Response usually sent back in JSON or XML.

-REST, RPC, and GraphQL are all request-response API architectures

Representational State Transfer Info 101

-REST. RESTful API.

-The most popular modern choice for API design. Used by Google, Stripe, Twitter, Github, etc.

-The key of REST is resources

-Resource - an entity that can be identified, named, and handled on the web. A REST API exposes resources for CRUD (Create, Read, Update, Delete) transactions.

-Resource example: customer, charge, balance, refund, payout

-Rest API Rules:

- Resources types are part of URLs. Ex. */users*
- Multiple subdomains of a resource URL exist to allow different CRUD interactions. Ex. */users* (get all users), and */users/12839* (GET, PUT, PATCH or DELETE user w/ id 12839)
- The same URL can take in different HTTP method requests to perform different actions:
 - Create - POST - Create a new resource
 - Read - GET - Get entity. Read only, so good for using cache on GET.
 - Update - PUT - Replace an existing resource entirely. PATCH - Partial update of existing resource.
 - Delete - DELETE - Delete existing resource
- Standard HTTP response status codes are also returned by server, to indicate success or failure of request. Code ranges:
 - 2xx - Success
 - 3xx - Resource moved
 - 4xx - Client side error (ex. bad param, requested resource does not exist, etc.)
 - 5xx - Server side error (ex. server unreachable)
- JSON has become the standard return for most REST API

-Example Request:

GET /v1/charges/ch_CWyutlXs9pZyfD

HOST api.stripe.com

Authorization: Bearer YNoJ1Yq64iCBhzfL9HNO00fzVrsEjtVl

-A resource that exists only within another resource is better represented and managed via separate endpoints.

Ex. *POST /repos/:owner/:repo/issues //create an issue*

More REST API

-Learning Resource: <https://restfulapi.net/>

-REST is not a protocol, but an architectural style.

-Guiding Principles of REST:

- Uniform Interface
- Client-Server
- Stateless
- Cacheable
- Layered System
- Code on Demand (optional)

REST Uniform Interface

-REST design is based on the principle of generality: API is designed free from unnatural restrictions and limitations (ex. year only goes up to 2000). Design API in uniform interface via various methodology.

-Identification of Resources - Interface must uniquely identify each resource involved in client-server interaction

-Manipulation of resources through representations - Resources should have uniform representations in server response. API consumers should use these representations to modify resource state on server.

-Self-descriptive messages - Each resource representation should carry enough info to describe how to process the message. It should also provide info on the additional actions the client can perform on the resource.

-Hypermedia as the engine of application station - Client should only have to initial URL of the application. The client application should drive all other actions with the use of hyperlinks.

-Users CRUD via HTTP methods and resource URIs (Uniform Resource Identifiers)

REST Client-server

-An enforcement of separation of concerns. The client and server can evolve independently, as the user interface is separated from back-end and data storage concerns.

-Server components can be scaled, modified, etc., with the interface staying the same. This allows client backwards compatibility.

REST Stateless

-Each request to the server must contain *all* info required to perform the desired action on the resource. The server cannot use previously stored context, in this.

-This requires the client to entirely keep session state.

REST Cacheable

-A response should implicitly or explicitly label itself as cacheable or non-cacheable. If response is cacheable, the client app gets the rights to reuse the response data later for equivalent requests (over a specified period).

REST Layered System

-Architecture is composed of hierarchal layers by constraining a components behavior. Each component can only see within the layer they are interacting with.

-Ex. MVC layering, DDD layering

REST Code on Demand

- Client functionality can extend by downloading and executing code in the form of applets or scripts.
- Can allow quick integration of public API into consuming system.

REST Resource

-An abstraction of information that is a named entity within the API system. Ex. a document, image, account, order, etc.

-Resource state at any time is known as *resource representation*, which includes the data, the metadata describing the data, the hypermedia links that can help clients transition data to desired next state.

-Resource model - APIs set of interlinked resources

-Resource Identifier - Unique ID for each REST resource

-Media Type - The data format of a representation. A specification for how a representation is to be processed.

-A RESTful API looks like hypertext. Every addressable unit of info in the API carries an address, either explicitly (ex. link or id attributes) or implicitly (ex. desired from the media type definition and representation structure).

-Hypertext (aka hypermedia) - the simultaneous presentation of information and controls. The information becomes the way through which the consumer performs actions.

REST Self-Descriptive

-Resource representation is self descriptive. The client should act based on the media type associated with the resource.

-Typically, just one media type is associated to a resource. The resource is a model in a domain.

REST Resource Example

```
//A blog post
{
  "id": 123,
  "title": "What is REST",
  "content": "REST is an architectural style for building web services...",
  "published_at": "2023-11-04T14:30:00Z",
  "author": {
    "id": 456,
    "name": "John Doe",
    "profile_url": "https://example.com/authors/456"
  },
  "comments": {
    "count": 5,
    "comments_url": "https://example.com/posts/123/comments"
  },
  "self": {
    "link": "https://example.com/posts/123"
  }
}
```

REST Resource Methods

-Any method you use to access a resource or modify it's state.

-Resource methods decide which methods to link to different HTTP methods. A resource method could implement for PATCH, instead of PUT, for example.

Rest != HTTP

-Rest is an architecture, based on the six principles defined earlier. HTTP methods are simply instructions to modify/place/retrieve data in a specific way.

Remote Call Procedure

-RPC. A simple API paradigm in which a client executes a block of code on another server. Clients pass a method name, receive back a response, typically in JSON or XML.

-REST = resources. RPC = actions.

-RPC Rules:

- An endpoint contains the name of the operation to be executed.
- API calls are made with the HTTP verb that is most appropriate for the request (ex. GET for read-only requests, POST for others)

-Work well for APIs that must expose a variety of actions that can be hard to implement with simple CRUD guidelines. Useful for actions that require work with many types of resources.

-Ex.

```
POST /api/conversations.archive
HOST slack.com
Content-Type: application/x-www-form-urlencoded
Authorization: Bearer xoxp-1650112-jgc2asDae
channel=C01234
```

-Action ex. archive, join, kick, leave, rename, postMessage, conversations.history, conversations.list

-RPC is not exclusive to HTTP protocol, with other options like Apache Thrift available.

GraphQL

-Another API paradigm. A query lang for APIs. Originally developed by FB in 2015, now used by Yelp, Pinterest, etc..

-Allows client to define the structure of the required data. Server then returns in exactly that structure.

-Query and response example:

```
//query
{
  user(login: "saurabhsahni") {
    id
    name
    company
    createdAt
  }
}

//response
{
```

```

    "data": {
      "user": {
        "id": "MDQ6VXNlcjY1MDI5",
        "name": "Saurabh Sahni",
        "company": "Slack",
        "createdAt": "2009-03-19T21:00:06Z"
      }
    }
  }
}

```

-Unlike REST and RPC, there is only one URL endpoint. You also do not use HTTP verbs to describe your operation. You can define either GET or POST, when call URL endpoint, but that is it, and rest is defined by request structure.

-Example API call:

```

POST /graphql
HOST api.github.com
Content-Type: application/json
Authorization: bearer 2332dg1acf9f502737d5e
xoxp-16501860787-17163410960-113570727396-7051650
{
  "query": "query { viewer { login }}"
}

```

GraphQL Advantages

-Saves multiple round trips - User can request data across multiple resources in a single request. Can improve speed, as such.

-Avoids versioning - Can add new fields and types without affecting existing queries. Can also perform easy deprecation of fields, by using log analysis, to figure which clients are using a field, and remove or deprecate when not being used.

-Smaller payload size - GraphQL is strongly typed. This helps insure syntactically correct queries.

-Introspection - GraphQL is natively discoverable, unlike REST, which requires native exploration tools, like Swagger. GraphQL Introspection is an in-browser IDE for exploration.

API Expert Advice

-Avoid payload creep by building more endpoints, instead of increasing your payloads. Data returned should be only what is needed.

GraphQL Disadvantages

-More complexity for the API provider. Server must perform extra processing to parse complex queries and verify params.

-Query optimization can be difficult.

-GraphQL query structure is not as widely understood as simple REST one.

Rest vs RPC vs GraphQL Summary

	REST	RPC	GraphQL
Overview	Exposes data as resources. Uses standard HTTP methods to represent CRUD operations.	Exposes action-based API methods - clients pass method names and args.	A query lang for APIs - clients define the structure of the response.
Service Ex.	Stripe, Google, Twitter	Slack, Flickr	FB, Github, Yelp
Call Ex.	<i>GET /users/<id></i>	<i>GET /users.get?id=<id></i>	<i>query (\$id: String!) { user(login: \$id) { name company createdAt } }</i>
HTTP Verbs Used	<i>GET, POST, PUT, PATCH, DELETE</i>	GET, POST	GET, POST
Pros	-Standard method names, args, format, & status codes -Utilizes HTTP features -Easy to maintain	-Easy to understand -Lightweight payloads -High performance	-Saves multiple calls -Avoids versioning -Smaller payloads -Strongly typed -Built-in introspection
Cons	-Big payloads -Requires multiple calls	-Discovery is difficult -Limited standardization -Can lead to function explosion	-Requires additional query parsing -Backend performance optimization difficult -Too complicated for a simple API
When to Use?	APIs doing CRUD-like operations	APIs exposing actions	APIs needing query flexibility and maintaining consistency

2.2 Event-Driven APIs

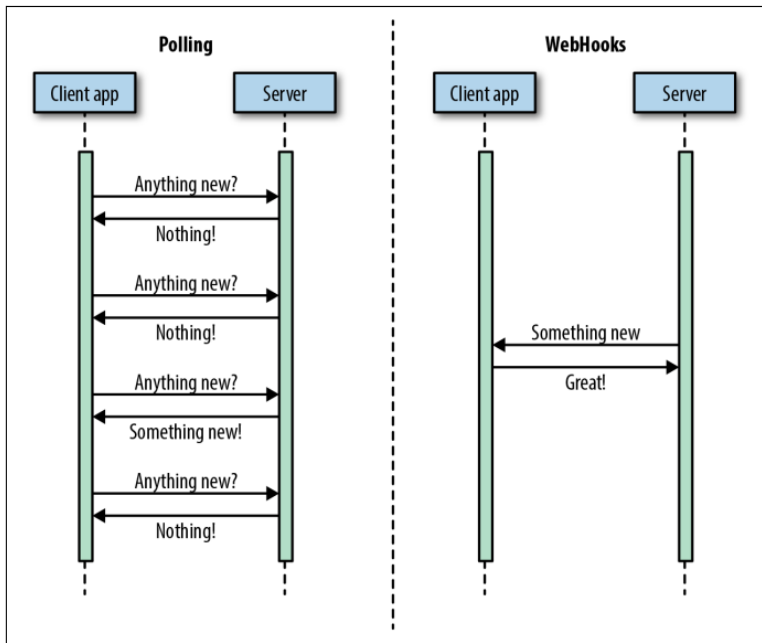
-For request-response APIs with quickly changing data, the response can quickly become stale. A *polling* API solves this. With a polling API, consumer services are constantly querying to API endpoints, at a pre-determined frequency, to be notified of new data.

-Three most common: WebHooks, WebSockets, and HTTP Streaming

WebHooks

-A URL that accepts an HTTP method (*POST, GET, PUT*, etc.). A message is sent to a URL when something happens.

-Useful for receiving real time updates. Ex. a new slack channel is created to. Update is POSTed to webhook. User is notified.



-Excellent for easily sharing real-time data from one service to another.

-As simple as creating a new HTTP endpoint to receive events. Can these easily integrate into existing infrastructure.

-Must ensure some requirements with webhooks

- Retries - To ensure WebHooks are delivered successfully, important to build a system that will retry WebHook delivery on errors. Can limit number of retries.
- Security - WebHook security is still evolving. Devs must be sure they are receiving a legit webhook and not an unverified one.
- Firewalls - If App is behind a firewall, can access APIs over HTTP, but receiving WebHooks may be difficult.
- Noise - Each WebHook represents a single event. When thousands of events occurring, all being sent by single WebHook, can be noisy.

WebSockets

-A protocol used to establish a two-way communication channel over a single TCP (Transport Control Protocol) connection.

-Typically used between a web client a server, though can also be server-to-server.

-Often used by real-time applications, such as Slack, which it uses to send messages, etc. to Slack clients. Slack also provides a WebSocket-based Real Time Messaging API to devs, so they can integrate Slack into their own applications.

-Ex. Blockchain uses to send real time notifications about transactions and blocks

-Low client to server communication overhead and good for fast, live streaming data, and long lived connections.

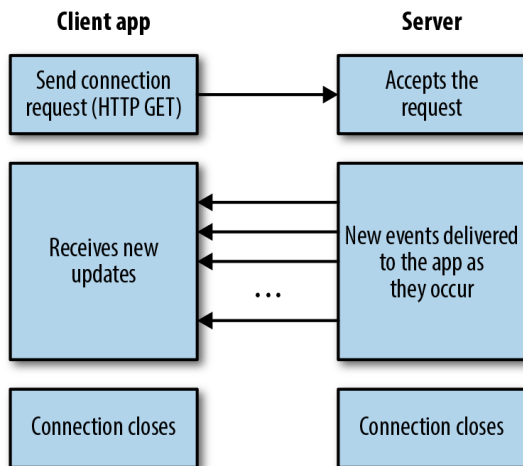
-WebSockets designed to work over port 80 or 443, enabling easy firewall integration of them.

-WebSockets require an open connection, so on Mobile, if signal is spotty, and connection dies, client will need to re-initiate it.

-WebSockets can scale poorly, as each connection requires it's own WebSocket, meaning, if 10k employees using Slack, and Slack integrated via WebSocket API, then there are 10k connections between Slack servers and integration app.

HTTP Streaming

-Done via HTTP request-response APIs, in which clients send an HTTP request and the server returns and HTTP response, where the response can be indefinite. A persistent connection.



-Server can create persistent connection in two ways:

- Set the *Transfer-Encoding* header to be chunked. This allows data to be continuously sent in newline delimited strings.
- Use SSE (server-sent events)

-Twitter uses the HTTP streaming protocol to deliver data through a single connection between an app and Twitter's streaming API. This requires new tweets to pass over without polling required.

-HTTP Streaming is easy to consume, though buffering can be an issue, where data might be rendered until a specific buffer level is met.

-If clients want to frequently change the events they are listening to, HTTP Streaming might also not be good, as it requires reconnection.

	WebHooks	WebSockets	HTTP Streaming
Overview	Event notification via HTTP callback	Two-way streaming connection over TCP	Long-live connection over HTTP
Service Ex.	Slack, Stripe, Github	Slack, Blockchain	Twitter, Facebook
Pros	-Easy server-to-server communication -Uses HTTP protocol	-Two-way communication -Native browser supports -Can bypass firewalls	-Can stream over simple HTTP -Native browser support -Can bypass firewalls
Cons	-Do not work across firewalls or in browsers	-Need to maintain a persistent connections	-Bi-directional comm is difficult

	-Handling failures, retries, security is hard	-Not HTTP	-Re-connections required receive diff events
When to Use?	To trigger the server to serve real-time events	For two-way, real-time comm between browsers and servers	For one-way comm over simple HTTP

2.3 Closing Thoughts

-Each API paradigm may work well for one situation, but poorly for another. You may need to support multiple paradigms, also. When choosing paradigms, ensure requirements and restraints are well known, first.